Course Project Report on

OKUBE-ENERGY

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY
by

Sanchit Srisha Udupa (231IT060)
Aditi Pandey (231IT003)

under the guidance of

Dr. Bhawana Rudra



DEPARTMENT OF INFORMATION TECHNOLOGY
NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA
SURATHKAL, MANGALORE - 575025

October, 2026

Okube-Energy: Fusing Network and Carbon Cost in a Custom Container Scheduler

Sanchit Srisha Udupa
Department of Information Technology
NIT-Surathkal
Surathkal, Mangalore, India
sanchit.231it060@nitk.edu.in

Aditi Pandey
Department of Information Technology
NIT-Surathkal
Surathkal, Mangalore, India
aditip.231it003@nitk.edu.in

***Abstract*—Container orchestrators place workloads mainly by resource availability and treat network topology and carbon cost as secondary. We present Okube-Energy, a scheduler extension for Okube, a custom Go orchestrator, that scores every candidate node on network cost and carbon cost in one pipeline. Its novelty lies in four things: a composable pipeline in which the network and energy stages toggle independently, so the default, network-only, energy-only and fused schedulers are ablations of one code path; a task-class-aware carbon weight that is raised for time-flexible work and lowered for latency-critical work; a carbon cost on a fixed scale, which treats a zero-carbon grid as a valid value and does not inflate differences between uniformly clean grids; and an evaluation that computes CO_2 from power curves learned from 1115 SPECpower_ssj2008 systems. On a seeded simulation of a three-zone cluster, with settings chosen on one set of seeds and reported on another, the fused scheduler at an energy weight of 1.5 cuts dynamic CO_2 by 12.9% (95% CI ± 1.2) against the default scheduler while keeping mean dependency latency 81% below default (network-aware alone: 83%). Under a saturating workload the carbon gain shrinks to 2.4%. The study is simulation-based and uses illustrative regional carbon intensities; its limits are stated in Section VIII.**

***Index Terms*—container orchestration, scheduler, network-aware scheduling, carbon-aware computing, power model, SPECpower, carbon intensity, Go**

I. Introduction

A container orchestrator decides which node runs each task. The default Kubernetes scheduler ranks nodes by resource fit: whether a node has enough CPU, memory and disk, and how balanced the resulting utilisation is. Two costs are largely absent from that model. The first is *network cost*: services that depend on each other pay inter-service latency when placed far apart, and tail latency degrades even when every node is well provisioned. The second is *carbon cost*: the same task draws different energy on different nodes, and the electricity behind each node has a different carbon intensity, so a resource-optimal placement is often not a carbon-optimal one.

Prior work treats these costs separately. Diktyo [1] makes placement network-aware but is carbon-blind. Piontek et al. [3] separate latency-critical from time-shiftable jobs and move the latter to lower-carbon windows, but have no network model and rely on a simple moving-average forecast. Saad et al. [2] predict per-workload energy with federated learning on top of Kepler [4], but stop at prediction and make no scheduling decision.

This paper builds the missing combination inside Okube, a custom manager/worker orchestrator written in Go with Docker execution and etcd coordination. We deliberately did not write a Kubernetes plugin: a custom scheduler lets us change placement logic end to end, and the same pipeline code is driven unchanged by an offline simulator.

A. Novelty and contributions

Table I positions this work against the three papers above. The contributions are:

1) **One pipeline, four schedulers.** Network scoring and energy scoring are independent stages of a single pipeline. The default, network-only, energy-only and fused schedulers differ only in two flags, so any difference between them is attributable to a stage and not to a different implementation. An energy weight of zero reproduces the network-only scores exactly, which a regression test enforces.
2) **Class-aware fusion.** The carbon weight is scaled by task class ($\times 0.35$ critical or urgent, $\times 1.4$ flexible). This adapts the critical/shiftable distinction of [3] from *when* to *where*, and combines it with a Diktyo-style [1] network cost.
3) **A carbon cost on a fixed scale.** Our first formulation min–max normalised the energy cost across the candidates. That stretches any spread to the full range, so a 15 g/kWh node looked as bad as it could be next to a 0 g/kWh node. Scoring against a fixed reference intensity removes this failure, and an explicit "unknown" replaces the sentinel that made a real zero look unset.
4) **Power curves learned from open data.** We scraped the SPECpower_ssj2008 published results (1118 systems with full load curves) and trained a monotone gradient-boosted model from hardware to watts. It replaces a made-up idle/max pair whose error was 65% by one with 12.0% held-out error.
5) **A reproducible, paired evaluation.** A seeded simulator drives the real scheduler code. Settings are chosen on one seed range and results reported on a disjoint one, with paired comparisons and confidence intervals.

Our headline finding is that the fused scheduler keeps nearly all of the network latency gain while delivering a carbon

reduction that the network-aware scheduler alone does not. We also report where that gain disappears and which parts of the comparison are not as clean as they first appear (Section VIII).

The rest of the paper is organised as follows. Section II reviews related work, Section III describes the architecture, Section IV the scoring model, Section V the data and power model, Section VI the evaluation method, Section VII the results, and Sections VIII–X the discussion, threats, conclusion and future work.

II. Background and Related Work

Network-aware scheduling. Diktyo [1] models an application as a dependency graph and places services to reduce latency and respect bandwidth, reporting large latency reductions against the default Kubernetes scheduler. We adopt its idea of costing a node by the latency to the nodes hosting a service's dependencies.

Carbon-aware scheduling. Piontek et al. [3] classify jobs as critical or shiftable and delay the latter into lower-carbon windows, using a weighted moving average to forecast carbon. A recent survey [5] covers the wider space of carbon-aware container scheduling. These systems choose *when* to run; we use the class distinction to choose *where*.

Energy prediction. Kepler [4] estimates container power from hardware counters. Saad et al. [2] extend it with federated XGBoost models trained on SPECpower data, reporting lower error than a centralised model, but their contribution ends with the prediction. We use the same model family and the same public benchmark family, as a single non-federated model.

Data sources. SPECpower_ssj2008 [6] publishes measured power at eleven load levels (active idle to 100%) for servers from many vendors. The UK Carbon Intensity API [7] publishes the carbon intensity of Britain's electricity by region, with a forecast. Section V describes how each is used.

III. System Architecture

Okube follows a manager/worker model. The manager keeps cluster state and the pending-task queue and runs the scheduler; workers run Docker containers and report resource statistics; etcd provides shared state and leader election. Figure 1 shows the whole system, including the offline power-model and evaluation path built for this work.

Pipeline. Scheduling is an ordered sequence of plugin stages mirroring the Kubernetes Scheduling Framework but implemented natively: (1) *queue sort*, a topological sort so that a service is placed after its dependencies; (2) a *resource filter*; (3) a *network filter* that rejects nodes violating a hard per-edge latency or bandwidth constraint; (4) a *resource score*; (5) a *network score*; (6) the *energy score* added in this work; and (7) selection of the lowest fused score. Stages 5 and 6 are skipped independently through a score context, which is what lets one implementation serve all four evaluated configurations.

Metadata. Tasks carry a criticality, a flexibility flag, an estimated duration and a deadline, which can be set in a deployment manifest. Nodes carry a carbon intensity, and the manager derives load from the number of running tasks and each worker's reported capacity. Carbon intensity is read from a JSON file keyed by worker; `scripts/fetch_uk_carbon.py` writes that file from the UK Carbon Intensity API.

IV. Scoring Model

For a task t and a candidate node n, scores are costs: lower is better.

Network cost. For a service s in an application group, the network cost of n is the summed latency to the nodes hosting s's dependencies,

$$\mathrm{cost}_{\mathrm{net}}(n) = \sum_{e \in \mathrm{deps}(s)} \mathrm{latency}\big(n, \mathrm{node}(e.\mathrm{to})\big). \tag{1}$$

Network and resource costs are each min–max normalised to $[0, 100]$ across the candidates and combined with weights 0.7 and 0.3:

$$\mathrm{base}(n) = 0.7\,\mathrm{norm}\big(\mathrm{cost}_{\mathrm{net}}(n)\big) + 0.3\,\mathrm{norm}\big(\mathrm{cost}_{\mathrm{res}}(n)\big). \tag{2}$$

Without topology or dependency data this stage is a no-op and the resource score passes through, so the scheduler degrades gracefully.

Carbon cost. Let c_n be the node's carbon intensity as a unitless factor ($\mathrm{gCO_2/kWh}/1000$) and $L_{n,t}$ a load multiplier,

$$L_{n,t} = 1 + \frac{\mathrm{mem}(t)}{\mathrm{mem}(n)} + \frac{\mathrm{cpu}(t)}{\mathrm{cores}(n)} + 0.08\,\mathrm{tasks}(n). \tag{3}$$

The energy cost is measured on a fixed 0–100 scale,

$$E(n,t) = 100\,\min\Big(1, \frac{c_n}{c_{\mathrm{ref}}}\Big)\,\min\Big(1, \frac{L_{n,t}}{L_{\mathrm{ref}}}\Big), \tag{4}$$

with $c_{\mathrm{ref}} = 0.3$ (300 g/kWh, about the dirty end of Great Britain's regional values) and $L_{\mathrm{ref}} = 2$. A node with unknown intensity falls back to a default factor of 0.5; a known intensity of 0 costs nothing.

Class-aware fusion. The carbon weight is a base weight w scaled by the task's class,

$$w_e(t) = w \times \begin{cases} 0.35 & \text{critical or urgent} \\ 1.4 & \text{flexible} \\ 1 & \text{otherwise,} \end{cases} \tag{5}$$

and the fused score, minimised over candidates, is

$$\mathrm{score}(n) = \mathrm{norm}\big(\mathrm{base}(n)\big) + w_e(t)\,E(n,t). \tag{6}$$

The incoming scores are normalised so that w means the same whether they came from the network stage or, without topology data, from raw resource costs.

Properties. (i) $w = 0$ returns the incoming scores untouched, so the network-only baseline is exact. (ii) Because E uses a fixed scale, three grids at 0, 4 and 15 g/kWh differ by less than 3 points, whereas the normalised form scored them 0, 27 and 100; a real spread of 0, 100 and 237 g/kWh differs by about 40 points at idle load. (iii) Intensities above c_{ref} saturate, so two very dirty grids tie and the scheduler does not chase differences that no longer matter. Each property is covered by a unit test.

TABLE I
POSITIONING AGAINST THE CLOSEST PRIOR WORK, BASED ON EACH PAPER'S STATED SCOPE.

	Network-aware	Carbon / energy signal	Job-class aware	Learned power model	Fused, with ablations
Diktyo [1]	✓	×	×	×	×
Piontek et al. [3]	×	✓(moving average)	✓(when)	×	×
Saad et al. [2]	×	✓(federated prediction)	×	✓	×
Okube-Energy (this work)	✓	✓(regional intensity)	✓(where)	✓(single model)	✓

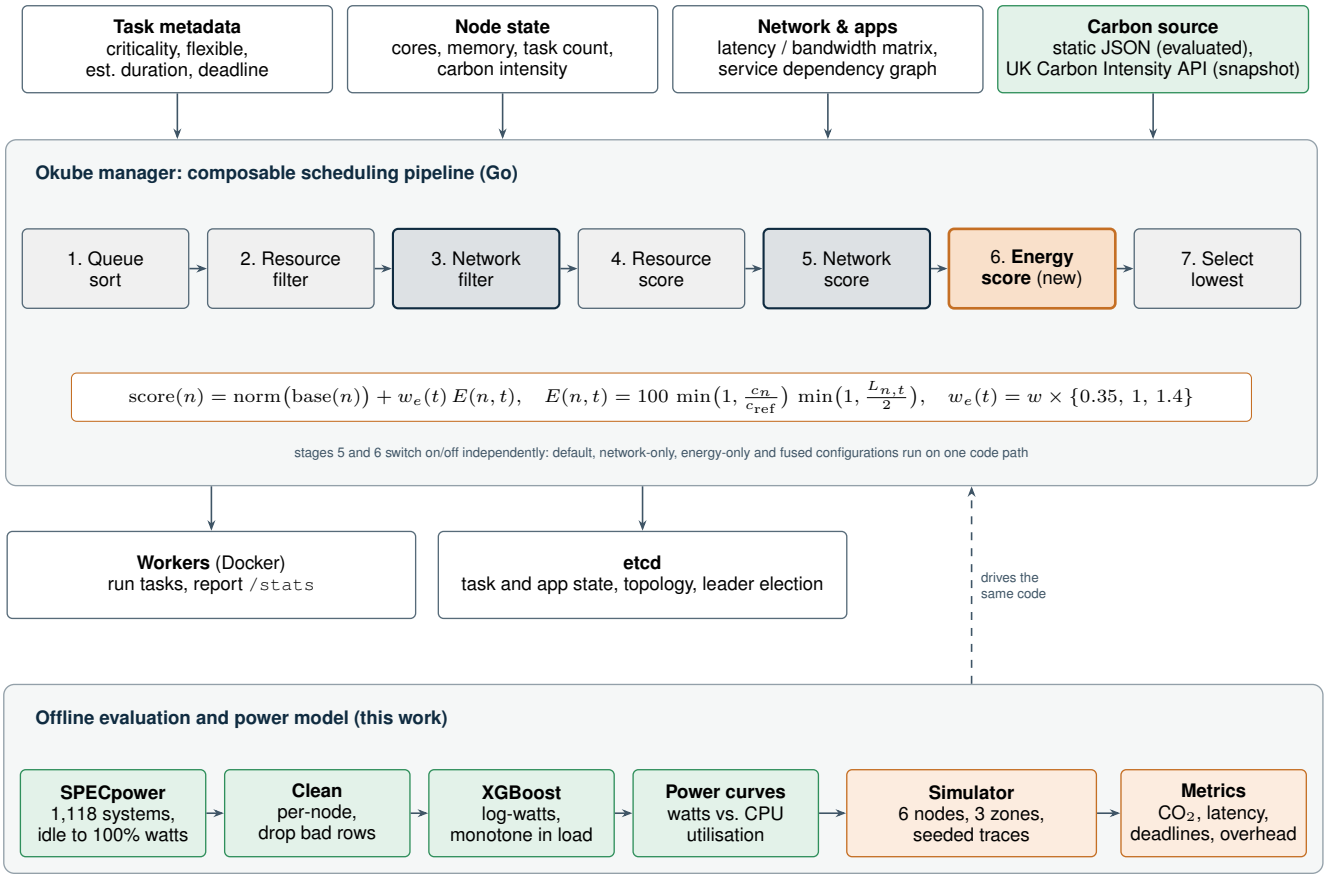


Fig. 1. Architecture of Okube-Energy. Four inputs feed a seven-stage scheduling pipeline in the manager; stages 5 and 6 (network and energy score) are independently switchable. The bottom panel is the offline path: SPECpower data is cleaned and used to train a power model, whose curves let the simulator compute CO_2 while driving the same pipeline code.

V. DATA AND POWER MODEL

A. SPECpower_ssj2008

SPEC publishes, for every accepted result, the hardware configuration and the average active power at ten target loads from 10% to 100% and at active idle. We read the public results summary and then each result's detail page, respecting the site's requested crawl delay, which gave 1118 systems with complete eleven-point curves (published 2007–2026). Systems with several nodes report whole-system power and hardware, so cores, chips, threads, memory and watts are divided by the node count. After dropping three systems with missing values (no system had idle power above full-load power or non-rising power), 1115 systems and 12,265 (system, load) rows remain.

B. Model

Utilisation is taken as the target load. The features are utilisation, chips, cores, threads, clock frequency, memory, publication year, cores per chip, threads per core and memory per core. We fit XGBoost [8] (400 trees, depth 4, learning rate 0.05, row subsampling 0.8) to $\log(\text{watts})$, with a monotone constraint that predicted power cannot fall as utilisation rises. Evaluation is by held-out *system*, so no system appears in both train and test, in two ways: five-fold grouped cross-validation, and a temporal split that trains on systems published up to 2022 (800) and tests on later ones (315).

Table II shows that the model reduces the error of the placeholder line from 65% to 12.0%. A straight line through each system's true idle and full-load watts has only 7.7% error,

TABLE II
HELD-OUT ERROR IN WATTS PER NODE (MAE) AND PERCENT (MAPE).

	Grouped CV		Temporal split	
Model	MAE	MAPE	MAE	MAPE
Placeholder line (100–250 W)	142.5	65.1	250.8	65.9
Line through true idle/max (oracle)	20.5	7.7	33.7	8.2
Ridge regression	75.7	32.9	205.1	46.2
XGBoost (ours)	32.7	12.0	118.0	25.8

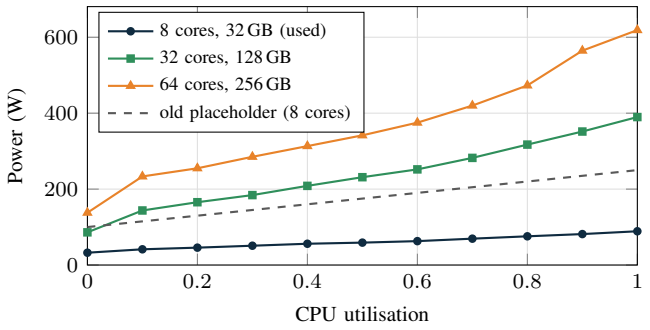


Fig. 2. Power curves predicted by the model for three hardware profiles, and the 100–250 W placeholder it replaces.

so a line between those two points is a good description of a real curve, and the extra error of the model comes from not knowing the idle and peak of a specific machine. Error on newer hardware is larger (26% on the temporal split). Figure 2 shows the predicted curves for the node types used here. For an 8-core single-socket node the model predicts 32 W idle, 59 W at half load and 89 W at full load, against a median of 20, 44 and 81 W for 34 comparable real systems from 2019 or later.

C. UK Carbon Intensity API

The Carbon Intensity API [7] is operated by NESO with the University of Oxford, EDF Europe and WWF, is licensed CC BY 4.0, and needs no key. Its regional endpoint returns the current half-hour's indicative intensity for each of Great Britain's regions (and national aggregates), and a forecast is available. On 23 October 2026 the regional values spanned 0 (North Scotland) to 237 g/kWh (South Wales). We use it in two ways. First, `scripts/fetch_uk_carbon.py` converts a regional snapshot into the `carbon_intensity.json` the manager loads, and a test checks that file loads with a real zero preserved; a snapshot taken on 24 October 2026 read 0, 5 and 137 g/kWh for North Scotland, North West England and South Wales. Second, those magnitudes calibrated the *illustrative* intensities of the simulated zones (Section VI). The simulation does not consume the live feed, and continuous refresh is future work.

VI. Evaluation Method

Simulator. A deterministic simulator drives the real scheduler pipeline (filter, score, select) with an event loop: arrivals are scheduled, tasks occupy cores and memory until they finish, a task waits if no node has room, and a service waits for its dependencies to be placed. It uses an offline, deterministic least-loaded resource scorer so that decisions do not depend on call order or on live worker statistics.

Scenario. Six nodes of 8 cores and 32 GiB in three zones of two nodes: `scot` (50 g/kWh), `nw` (180) and `south` (260). Latency is 2 ms within a zone and 12, 10 and 22 ms between `scot–nw`, `nw–south` and `scot–south`. The cleanest zone is thus the farthest from the dirtiest, so latency-driven and carbon-driven placements disagree. The intensities are illustrative UK-like values, not measurements. Every node uses the learned 8-core curve.

Workloads. A trace is a seeded sequence of application deployments drawn from three templates (a four-service shop, a three-service analytics job and a three-service media pipeline). The *default* workload has 60 deployments with a mean inter-arrival of 75 s, sized for about 35% average utilisation, so that nodes have room and the scheduler has a choice. Each deployment is critical, flexible or default with probability 0.2, 0.4 and 0.4, runs 60–600 s, and has a deadline 1.5, 6 or 3 times its duration after arrival. The *stress* workload arrives 2.5 times faster and saturates the cluster.

Configurations and metrics. The four configurations are default (stages 5 and 6 off), network-only (6 off), energy-only (5 off) and fused. Network-off configurations also withhold the topology from the hard network filter, so they are genuinely network-blind. We report (i) *dynamic* CO_2: each node's power above idle, read from its curve at the actual utilisation and integrated over time, shared among the running tasks by CPU request, and multiplied by the node's intensity; (ii) mean *dependency latency*, Eq. (1) for services that have dependencies; (iii) the deadline-violation rate; and (iv) the scheduling time per decision. Idle power is excluded from CO_2 because no placement avoids it.

Protocol. Within a seed every configuration sees the identical trace, so differences are paired. We chose the energy weight on seeds 1–30 as the largest w for which mean latency stays within 10% of network-only and the violation rate does not rise on both workloads, which gives $w = 1.5$. All results below come from the disjoint seeds 101–150 (50 runs per cell). Intervals are 95% confidence intervals on the paired mean, $1.96\, s/\sqrt{n}$.

VII. Results

Table III gives the main comparison. At the scheduler's default weight of 0.2 the fused scheduler already cuts CO_2 by 5.2% against the default scheduler while network-only cuts it by only 1.7%. At the selected weight of 1.5 the fused scheduler cuts it by 12.9% ($\pm$1.2) and keeps latency 81% below default, close to the 83% of network-only. The energy-only scheduler at the same weight reaches 7.3% with a latency reduction of only about 20%.

Trade-off. Figure 3 traces the weight sweep. The fused curve stays far to the left of the energy-only curve: for the same carbon reduction it pays a fraction of the latency. Raising w further buys more carbon (about 35% at $w = 5$) for

TABLE III

DYNAMIC CO_2 (G PER RUN, MEAN OVER 50 SEEDS) AND MEAN DEPENDENCY LATENCY. Δ IS THE PAIRED CHANGE AGAINST THE DEFAULT SCHEDULER; CO_2 CHANGES CARRY A 95% CONFIDENCE INTERVAL. ROWS BELOW THE RULE USE THE SELECTED WEIGHT.

Default workload

Config	w	CO_2	ΔCO_2 (%)	Lat. (ms)	ΔLat. (%)
Default	–	25.66	–	13.62	–
Network-only	–	25.20	-1.7 ± 0.9	2.26	-83.3
Energy-only	0.2	24.91	-2.8 ± 0.8	11.82	-12.9
Fused	0.2	24.32	-5.2 ± 1.1	2.09	-84.6
Energy-only	1.5	23.77	-7.3 ± 0.8	10.84	-20.2
Fused	1.5	22.34	-12.9 ± 1.2	2.57	-81.0

Stress workload

Config	w	CO_2	ΔCO_2 (%)	Lat. (ms)	ΔLat. (%)
Default	–	24.51	–	12.99	–
Network-only	–	24.48	-0.1 ± 0.6	5.62	-56.5
Energy-only	0.2	24.36	-0.6 ± 0.5	12.01	-7.3
Fused	0.2	24.24	-1.1 ± 0.6	5.52	-57.2
Energy-only	1.5	23.97	-2.2 ± 0.6	11.17	-13.6
Fused	1.5	23.91	-2.4 ± 0.7	5.98	-53.7

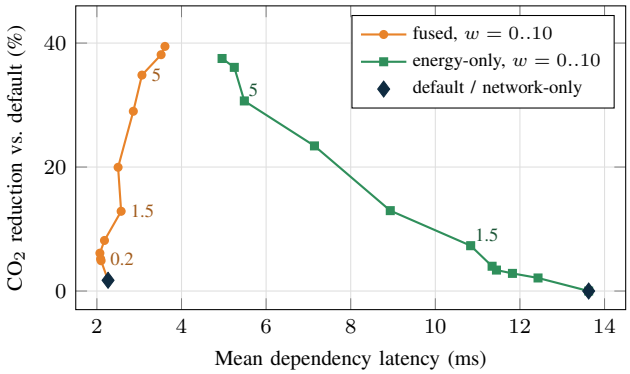


Fig. 3. Carbon–latency trade-off over the energy weight w (default workload; labels give w). Diamonds mark the default scheduler (right) and network-only (left).

TABLE IV

MEAN GRID INTENSITY (G/KWH) OF THE NODES USED, BY TASK CLASS (DEFAULT WORKLOAD).

Config	Flexible	Critical
Default	162	161
Network-only	161	164
Energy-only (w=1.5)	133	170
Fused (w=1.5)	119	183

progressively more latency, which is the trade-off the weight exposes, and our 10% latency budget is a judgement: a looser budget would select a larger weight.

Task classes. Table IV shows the mean grid intensity of the nodes each class ran on. With class-aware weights, flexible tasks move to cleaner nodes (119 g/kWh at the selected weight, against 162 for the default scheduler), while critical tasks move the other way (183 g/kWh) because flexible work takes the clean capacity.

Stress workload. When the cluster is saturated only one or two nodes have room for each task, and the scheduler has little to choose. Carbon gains fall to 2.4% at the selected weight, and the latency reduction is smaller (54% below default, against 56% for network-only).

Deadlines and overhead. Deadline-violation rates do not differ between configurations (about 0.03% on the default workload and 2.4–2.6% under stress), so energy-awareness at these weights does not cost deadlines, but our deadlines do not separate the schedulers either. A fused scheduling decision takes about 3.4 μs with the offline scorer, against under one microsecond for the default scheduler; with the EPVM scorer it would be dominated by its worker-statistics calls.

VIII. DISCUSSION AND THREATS TO VALIDITY

The comparison with energy-only is not a pure test of fusion. In the fused scheduler the resource term is weighted 0.3 by the network stage; in energy-only, with the network stage off, the resource score passes through unweighted. The energy term therefore competes against a weaker load-balancing term in the fused scheduler, and part of fused's lead over energy-only is this reweighting and not synergy between network and carbon. A like-for-like comparison would give energy-only the same resource weight.

The learned model does not influence placement. Scheduling decisions use the heuristic of Eq. (4); the power model only converts placements into CO_2. We therefore do not claim a scheduler driven by predicted energy, and only that CO_2 is computed from a model grounded in measured data.

Carbon data are illustrative, and static. Zone intensities are UK-like magnitudes, not measurements, and are constant through each run. Absolute CO_2 is a relative comparison between schedulers, not a real emission. Static intensities also remove the largest real lever, running work when the grid is clean, which we do not implement.

Power model limits. A single non-federated model; identical hardware in every node, so the scheduler cannot exploit efficiency differences; target load stands in for CPU utilisation; and error of 12.0% on held-out systems and 26% on newer ones. Systems with the same chips and cores differ by factors of two or three in watts, which no feature here explains.

Simulation scope. Latency is the topological cost of Eq. (1), not measured request latency; the weight was tuned and the results reported on different seeds but the *same* scenario and workload generator, so generalisation to other clusters is untested; and no live cluster run backs the numbers.

Side effects. At high weights critical tasks end on dirtier nodes (Table IV); and dynamic CO_2 excludes idle power, which is 63% of total CO_2 on the default workload (46% under stress). Consolidating work so idle nodes can be powered down is a lever we do not model.

IX. Conclusion

We extended Okube with a single scheduling pipeline in which network and carbon costs are independent, class-aware stages. A fixed-scale carbon cost, a power model learned from 1115 real systems, and a paired, held-out evaluation support a specific claim: on a three-zone simulated cluster the fused scheduler at an energy weight of 1.5 reduces dynamic CO_2 by 12.9% over the default scheduler while keeping latency 81% below default, a combination neither the network-only nor the energy-only scheduler reaches. The claim comes with clear limits: illustrative carbon data, one hardware type, a saturated-load regime where the gain largely vanishes, and a comparison to energy-only that is partly a reweighting effect.

X. Future Work

- **Live carbon feed.** Refresh intensities continuously from the UK Carbon Intensity API, keeping the static file as an offline fallback, and evaluate on a multi-region deployment.
- **Temporal placement.** Use the API's forecast to delay flexible jobs with deadline slack into cleaner windows, adding *when* to *where*.
- **Predicted energy in the score.** Feed the marginal watts of a placement from the power model into Eq. (4), which also matters once nodes differ in hardware.
- **Federated power model.** Train per-partition models and merge them, as in [2], and compare with the single model here.
- **Fair ablation and live runs.** Give energy-only the same resource weight as fused, and validate the simulator against a real multi-worker cluster.
- **Kubernetes port.** Reimplement the filter and score stages as a Scheduling Framework plugin once the design is stable.

Acknowledgment

The authors would like to express their sincere gratitude to the Department of Information Technology, NITK Surathkal, for providing continuous support and guidance throughout this project, and to our guide, Dr. Bhawana Rudra. We also thank our mentors and peers for their valuable feedback during the development phase.

References

[1] J. Santos, C. Wang, T. Wauters, and F. De Turck, "Diktyo: Network-aware scheduling in container-based clouds," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4461–4477, 2023, doi: 10.1109/TNSM.2023.3271415.

[2] Z. Saad, J. Yang, H. Leung, and S. Drew, "Towards carbon-aware container orchestration: Predicting workload energy consumption with federated learning," in *Proc. IEEE Smart World Congress (SWC)*, 2025, arXiv:2510.03970.

[3] T. Piontek, K. Haghshenas, and M. Aiello, "Carbon emission-aware job scheduling for Kubernetes deployments," *J. Supercomput.*, vol. 80, pp. 549–569, 2024, doi: 10.1007/s11227-023-05506-7.

[4] M. Amaral *et al.*, "Kepler: A framework to calculate the energy consumption of containerized applications," in *Proc. IEEE Int. Conf. Cloud Computing (CLOUD)*, 2023.

[5] "A survey on task scheduling in carbon-aware container orchestration," arXiv:2508.05949, Aug. 2025.

[6] K.-D. Lange, "Identifying shades of green: The SPECpower benchmarks," *Computer*, vol. 42, no. 3, pp. 95–97, 2009. Results: https://www.spec.org/power_ssj2008/results/, accessed Oct. 2026.

[7] National Energy System Operator (NESO) with the University of Oxford, EDF Europe and WWF, "Carbon Intensity API," https://carbonintensity.org.uk, CC BY 4.0, accessed Oct. 2026.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.